

TP1 — Sistemas Digitales

Respuestas Ejercicios 1, 2, 3 y 6

Integrantes

- [Gomez, Tomas] [tomas.w.gomez@gmail.com][1170/23]
- Segade, Santiago Ariel][segadesantiago7@gmail.com][552/25]
- [Bermeo Morales Santiago][Santumorales11@gmail.com][481/25]

1. Ejercicio 1 — ALU y flags

1.1. Desarrollo de las cuentas

1.1.1. $10 + 20$

$$\begin{array}{r} 0000_000A \quad (10) \\ +0000_0014 \quad (20) \\ \hline 0000_001E \quad (30) \end{array}$$

Flags:

- $Z = 0$: el resultado no es cero.
- $N = 0$: el bit más significativo es 0.
- $C = 0$: no salió un acarreo de los 32 bits.
- $V = 0$: no hay overflow, ya que 30 se puede representar con signo.

1.1.2. $10 - 20$

$$20 = 0000_0014 \quad - 20 = FFFF_FFEC$$

$$\begin{array}{r} 0000_000A \\ +FFFF_FFEC \\ \hline FFFF_FFF6 \end{array}$$

FFFF_FFF6 = -10 en complemento a dos.

Flags:

- $Z = 0$: el resultado no es cero.

- $N = 1$: el bit más significativo es 1.
- $C = 1$: en esta resta se produce préstamo.
- $V = 0$: -10 se puede representar con 32 bits con signo.

1.1.3. $10 - 10$

$$10 = 0000_000A \quad - 10 = FFFF_FFF6$$

$$\begin{array}{r} 0000_000A \\ +FFFF_FFF6 \\ \hline 0000_0000 \end{array}$$

El acarreo que queda fuera de los 32 bits se descarta.

Flags:

- $Z = 1$: el resultado es cero.
- $N = 0$: el bit más significativo es 0.
- $C = 0$: no se produce préstamo.
- $V = 0$: el resultado 0 se puede representar con signo.

1.1.4. $FFFF_FFFF + 1$

$FFFF_FFFF = -1$ en complemento a dos.

$$\begin{array}{r} FFFF_FFFF \\ +0000_0001 \\ \hline 1_0000_0000 \end{array}$$

El bit que queda en la posición 32 se descarta, por lo que el resultado es:

$$0000_0000$$

Flags:

- $Z = 1$: el resultado es cero.
- $N = 0$: el bit más significativo es 0.
- $C = 1$: hubo un acarreo fuera de los 32 bits.
- $V = 0$: $-1 + 1 = 0$, por lo que no hay overflow.

1.1.5. $7FFF_FFFF + 1$

$7FFF_FFFF = 2^{31} - 1$, el mayor número positivo representable.

$$\begin{array}{r} 7FFF_FFFF \\ +0000_0001 \\ \hline 8000_0000 \end{array}$$

Flags:

- $Z = 0$: el resultado no es cero.

- $N = 1$: el bit más significativo pasó a ser 1.
- $C = 0$: no hubo acarreo fuera de los 32 bits.
- $V = 1$: se sumaron dos números positivos y el resultado aparece como negativo.

1.1.6. $8000_0000 - 1$

$$8000_0000 = -2^{31}.$$

$$1 = 0000_0001 \quad -1 = FFFF_FFFF$$

$$\begin{array}{r} 8000_0000 \\ +FFFF_FFFF \\ \hline 1_7FFF_FFFF \end{array}$$

Se descarta el bit 32 y queda:

$$7FFF_FFFF$$

Flags:

- $Z = 0$: el resultado no es cero.
- $N = 0$: el bit más significativo es 0.
- $C = 0$: no se produce préstamo.
- $V = 1$: el resultado matemático sería $-2^{31} - 1$, que no se puede representar con 32 bits con signo.

1.2. Tabla

Operación	Resultado (hex)	Z	N	C	V
$10 + 20$	0000_001E	0	0	0	0
$10 - 20$	FFFF_FFF6	0	1	1	0
$10 - 10$	0000_0000	1	0	0	0
$FFFF_FFFF + 1$	0000_0000	1	0	1	0
$7FFF_FFFF + 1$	8000_0000	0	1	0	1
$8000_0000 - 1$	7FFF_FFFF	0	0	0	1

2. Ejercicio 2 — Banco de registros

2.1. Tabla

Se parte de $R1 = 10$, $R2 = 20$, con una lectura ya hecha con $A = 1$ y $B = 2$, y las escrituras deshabilitadas salvo cuando se indica.

Instante / acción	dout A	dout B	R2
Situación inicial	10	20	20
A cambia a índice 2, antes de que suba el reloj	10	20	20
Después de que sube el reloj	20	20	20
Después de escribir 99 en R2 por A	20	20	99
Después de que sube el reloj otra vez, sin escritura	99	99	99

2.2. Explicación sobre R0

En `reg_file.sv` hay dos protecciones que implementan R0.

En

```
regA_dout <= (regA_idx == '0) ? '0 : rf[regA_idx];
```

Si el índice es 0, la salida se fuerza a 0, sin importar qué haya guardado en $rf[0]$.

En la escritura:

```
if (regA_we && (regA_idx != '0)) rf[regA_idx] <= regA_din;
```

Si el índice es 0, la escritura directamente no se hace. $rf[0]$ nunca cambia.

El testbench lo verifica intentando escribir 123 y 456 en R0 por ambos puertos, y después comprobando que `dout A` y `dout B` siguen valiendo 0:

```
@(negedge clk); regA_idx=0; regB_idx=0; regA_we=1; regB_we=1;
regA_din=123; regB_din=456;
tick();
if (regA_dout != 0 || regB_dout != 0)
    $fatal(1, "R0 debe leerse como cero");
```

En resumen, R0 se comporta como una constante cero: siempre se lee 0 y cualquier intento de escritura se descarta. Esto lo hace útil como constante cero en operaciones y como destino “basura” para tirar resultados que no se quieren guardar.

3. Ejercicio 3 — Camino de datos

3.1. $R5 \leftarrow R1 + R2$

Se parte de $R1 = 10$ y $R2 = 20$. Se realiza la operación:

$$R5 \leftarrow R1 + R2$$

$$10 + 20 = 30$$

El resultado es 30, por lo que después de la escritura:

$$R5 = 30$$

Los registros de origen no se modifican:

$$R1 = 10, \quad R2 = 20$$

3.2. $R6 \leftarrow R5 - R1$

Se utiliza el resultado de la operación anterior:

$$R6 \leftarrow R5 - R1$$

$$30 - 10 = 20$$

Luego de la escritura:

$$R6 = 20$$

Los registros utilizados como origen mantienen sus valores:

$$R5 = 30, \quad R1 = 10$$

3.3. Actualización de operandos y escritura del destino

En la traza, primero se seleccionan $rs1$ y $rs2$ y se realiza la lectura del banco de registros. En el siguiente flanco se actualizan los operandos que recibe la ALU.

Una vez calculado el resultado, se habilita **rf_we** y se selecciona rd . En el siguiente flanco se escribe el resultado en el registro destino.

Para la primera operación:

$$R1 = 10, \quad R2 = 20 \quad \longrightarrow \quad ALU = 30 \quad \longrightarrow \quad R5 = 30$$

Para la segunda:

$$R5 = 30, \quad R1 = 10 \quad \longrightarrow \quad ALU = 20 \quad \longrightarrow \quad R6 = 20$$

Los registros de origen no cambian porque la escritura se realiza únicamente sobre rd . Si rd coincide con uno de los registros de origen, ese registro sí se modifica al realizar la escritura.

3.4. Por qué seleccionar *rd* no cambia inmediatamente el operando A

En `datapath.sv` se utiliza:

```
assign regA_idx = rf_we ? rd : rs1;
```

Cuando `rf_we` pasa a 1, `regA_idx` cambia de *rs1* a *rd*. Sin embargo, la lectura del banco de registros es síncrona, por lo que el valor de `rf_rs1_data` no cambia en ese momento.

El operando A de la ALU conserva el valor leído anteriormente hasta el siguiente flanco ascendente. Por eso seleccionar *rd* no cambia inmediatamente el resultado de la ALU y se puede escribir el resultado calculado en el registro destino.

4. Ejercicio 6 — Unidad de ejecución completa



Figura 1: Trazas de $R5 \leftarrow R1 + R2$. Señales mostradas: `clk`, `start`, `ready`, `done`, `present_state`, `capture_en`, `rf_we`, `regA_idx`, `rf_rs1_data` y `result`.

4.1. Flancos señalados

- **IDLE:** cuando `capture_en=1` en el estado IDLE, la orden se guarda en el registro de orden. Se capturan $rs1_q = 1$, $rs2_q = 2$, $rd_q = 5$ y $op_q = \text{ADD}$.
- **FETCH_OPS:** cuando `regA_idx=01`, el banco de registros lee $R1 = 10$ y lo entrega en `rf_rs1_data`.
- **WRITEBACK:** cuando `rf_we=1` y `regA_idx=05`, el banco escribe el resultado `result=30` en $R5$ al llegar el pulso del reloj.

4.2. Valores de `regA_idx` y `result` por estado

Estado	<code>regA_idx</code>	<code>result</code>
FETCH_OPS	$rs1_q = 1$	0x00000001e
EXECUTE	$rs1_q = 1$	0x00000001e
WRITEBACK	$rd_q = 5$	0x00000001e

`regA_idx` vale $rs1_q$ durante FETCH_OPS y EXECUTE porque `rf_we=0` y el mux del índice A selecciona la fuente. En WRITEBACK vale rd_q porque `rf_we=1` y el mux selecciona el destino.

`result` vale 30 en los tres estados. La ALU es combinacional: en cuanto los operandos están presentes, el resultado queda estable, y no cambia porque el registro de orden mantiene fijos $rs1_q$, $rs2_q$ y op_q .

4.3. ¿Por qué el resultado se obtiene antes de escribirse?

Porque la ALU es combinacional y la escritura al banco es síncrona.

1. En cuanto `rf_rs1_data=10` y `rf_rs2_data=20` están presentes, la ALU propaga `result=30` sin esperar ningún pulso del reloj.
2. El registro de orden mantiene $rs1_q$, $rs2_q$ y op_q estables durante FETCH_OPS, EXECUTE y WRITEBACK, así que el resultado no se recalcula con otros valores.
3. La escritura al banco solo ocurre en WRITEBACK, cuando la FSM activa `rf_we=1` y llega el pulso del reloj.